

Products and
TechnologyGroup
sDeveloper
sPartner
sEvent
sHelp
CenterTopic
PagesExplore
SAP

SAP Community / Developers / Artificial Intelligence / Blogs Posts / The RAG Chunking Strategies that can actually surv...

Artificial Intelligence Blogs Posts

AI Search



Search

The RAG Chunking Strategies that can actually survive Production

**Mohan-Sharma**
Product and Topic Expert[Sign In to Subscribe](#)

3 weeks ago



4 Likes



607

SAP MANAGED TAGS

Artificial Intelligence

SAP AI Core

Agentic AI

The Chunking Strategies That Actually Survive Production

If your RAG quality has plateaued, the bottleneck is almost never your vector DB or your reranker. **It is almost always your chunker.** The naive splitters in every quickstart tutorial destroy the one thing retrieval depends on: **context**.

This is a tour of the chunking strategies you will actually meet in production. For each one I cover how it works, why you would reach for it, why you would not, and where it fits. I am avoiding hard benchmark numbers on purpose - the field moves fast and figures age badly. What does not age is **the mechanism each strategy relies on, and the corpus shape it was built for.**

Why Chunking Is The Hidden Lever

A retriever can only return what your indexer produced. If your chunks are mid-sentence fragments stripped of context, no embedder, reranker, or LLM downstream will recover what you destroyed at ingest. Most teams over-invest in retrieval and **under-invest in chunking**. The fix is *upstream*.

Two failure modes show up over and over:

1. **The fragment problem.** A chunk reading "set the timeout to 30 seconds in the advanced settings tab" is meaningless without knowing what product, what version, and which settings tab.
2. **The boundary problem.** A chunk that ends mid-table, mid-code-block, or mid-sentence has lost the structural cue the author wrote precisely to communicate meaning.

Every strategy below is a different answer to those two problems.

Strategy 1: Fixed-Size Chunking

Workflow. Cut the document every N characters or N tokens, optionally with M characters of overlap. No grammar, no structure, no embeddings. Pure counting.

Example. A document about Kubernetes controllers gets sliced every 512 characters. One chunk ends with "the controller watches for chan" and the next begins with "ges and reconciles state". **Both chunks are now noise.**

Pros.

- Trivial to implement.
- Predictable token budget per chunk.
- Cheapest possible ingest.

Cons.

- Splits mid-sentence and mid-word.
- Breaks back-references. A chunk that says "the controller does X" no longer tells you which controller, because the noun it referred to is in a different chunk.
- Splits tables, code, equations down the middle.
- Embeddings get noisy because chunks are not self-contained.

When to use. As the baseline you measure improvement against. Acceptable if your corpus is short, uniform, and pre-cleaned (chat transcripts where each turn stands alone, log lines with fixed structure).

When NOT to use. Anywhere else. Treat it as a demo strategy. If a tutorial proposes this for a production RAG, the tutorial is wrong.

Strategy 2: Recursive Character Splitting

Workflow. Try the most semantic separator first (a blank line between paragraphs), fall back to weaker ones (single newline, space, character-level splits) if chunks remain too large. LangChain's `RecursiveCharacterTextSplitter` default separator list contains exactly four entries: paragraph break, line break, space, and an empty-string fallback.

How it works. If a paragraph fits within your chunk size, keep it whole. If not, drop down to sentences. If a sentence is still too long, drop to words. The hierarchy of separators is your knob.

Example. A blog post with clean paragraph breaks chunks naturally on the blank lines and you get clean paragraph chunks. A dense PDF dump with no paragraph breaks falls all the way through to space-splitting and you get fragments again.

Pros.

- Cheap, fast, deterministic.
- Respects paragraph and sentence boundaries when the document has them.
- The "no-thought" choice that mostly works as a starting point.

Cons.

- Document-blind. Does not understand that `## Section Header` matters more than a regular newline.
- Will still split inside tables and code unless you customise separators.
- Treats every newline as equal weight.

When to use. Your default starting point on a new corpus. Run it before reaching for anything fancier. Strong baseline against which everything else must justify itself.

When NOT to use. As a permanent answer for structured documents (Markdown, HTML, code, layout-rich PDFs). It cannot exploit structure it does not see.

Strategy 3: Sentence-Based Splitting

Workflow. Use a real linguistic tokenizer (NLTK Punkt, spaCy) to detect sentence boundaries, then group sentences until you hit chunk size.

Why it matters. Naive splitting on the period character breaks "Dr. Smith arrived at 3 p.m. The talk lasted 45 min." into garbage. A real sentence tokenizer keeps the abbreviations, decimals, and ellipses intact.

Example. A legal contract: "Pursuant to Section 4.2, the Party shall remit payment to Acme Inc. within 30 days." Naive split breaks this into four pieces. NLTK Punkt keeps it whole.

Pros.

- Handles abbreviations, decimals, ellipses without breaking mid-sentence.
- Multilingual via spaCy models.
- Cleaner inputs to downstream pipelines (sentence-window, propositions).

Cons.

- Slower than character splitting (the sentence model has overhead).
- Still document-blind. Ignores headings, tables, lists.
- No help on code, JSON, HTML.

When to use. Prose-heavy corpora where abbreviation handling matters: legal text, narrative content, support tickets, transcripts.

When NOT to use. Code, structured data, or anywhere the structure carries more meaning than the prose.

Strategy 4: Document Structure-Aware Splitting

Workflow. Parse the document's native structure (Markdown headings, HTML tags, PDF layout, code AST) and split on the boundaries the author already wrote.

How it works. A `MarkdownHeaderTextSplitter` walks `#`, `##`, `###` and treats each section as a chunk. A PDF parser like `Unstructured.io` detects layout (titles, lists, tables, captions) and chunks by element type. A code splitter parses the AST and keeps functions and classes together.

Example. A pricing comparison table in a product doc stays as one chunk instead of being cut after row three. A Python class with five methods stays whole instead of being split mid-method. A `## Authentication` section in an API doc stays self-contained, with its heading available as metadata for filtering.

Pros.

- Tables stay intact - massive win for technical reference content.
- Code functions and classes stay coherent.
- Headings can be indexed as metadata for filtering and reranking.
- Preserves what the author meant to be a unit.

Cons.

- Requires a parser per format. PDF parsing is hard, OCR is harder.
- Layout-rich PDFs need heavyweight tools (`Unstructured.io` hi-res, `LlamaParse`, `Docling`) which are slow and sometimes expensive.
- Bad input markdown (no consistent heading levels) means bad chunks.

When to use. *Always*, as the bottom layer of your pipeline. Stack other strategies on top, **never replace this floor**. The single highest-ROI thing you can do for technical documentation, product manuals, regulatory filings, and code-heavy corpora.

When NOT to use. When your input is unstructured prose with no markup signal (raw transcripts, chat logs, OCR'd notes). Use sentence-based or recursive instead.

Strategy 5: Hierarchical (Parent-Child / Small-to-Big)

Workflow. Build a tree: document, then sections, then paragraphs. Embed the small leaves (paragraphs). At retrieval time, if multiple leaves under the same parent match the query, return the parent instead of the individual leaves. `LlamaIndex` calls this auto-merging.

How it works. You **decouple the retrieval surface** (small chunks, high precision) **from the generation surface** (larger chunks, full context). The retriever finds tight matches; the merger expands them upward when the matches cluster.

Example. API documentation. The query "how do I authenticate" matches three paragraphs under the `## Authentication` section. Instead of returning three disjoint paragraphs, the retriever returns the whole section. The LLM gets coherent context; the user gets a coherent answer.

Pros.

- Solves the precision-vs-context tradeoff explicitly.
- Tight queries get tight chunks; broad queries get whole sections.
- Decouples retrieval surface from generation surface, which is the right architectural call.

Cons.

- More moving parts: tree storage, merge thresholds.
- Storage roughly doubles or triples (you index leaves and store parents).
- The merge threshold is a hyperparameter that needs evaluation.

When to use. Long documents with hierarchy: technical manuals, RFCs, regulatory filings, books, multi-section product docs. Anywhere the author already wrote a hierarchy and the query mix spans both narrow and broad questions.

When NOT to use. Short flat documents (tweets, FAQ entries, support tickets). There is no hierarchy to exploit and the added complexity buys you nothing.

Strategy 6: Semantic Chunking (the paper tiger)

Origin. Popularised by Greg Kamradt in his "5 Levels of Text Splitting" tutorial. Reimplemented as `SemanticChunker` in LangChain Experimental and `SemanticSplitterNodeParser` in LlamaIndex.

Workflow. Embed each sentence (or a sliding window of sentences). Walk the document; when adjacent-sentence cosine distance jumps above a threshold, insert a chunk boundary. The intuition is appealing: chunks should match topic boundaries, not character counts.

How it actually behaves in production.

- **Sentence embeddings are noisy on technical text.** Vocabulary repeats across topics. The embedder sees the same words and reports high similarity even when the meaning has shifted. The signal it is supposed to fire on barely exists.
- **The threshold is a knob nobody tunes.** Percentile, standard deviation, interquartile range - each has a different default and most teams ship the default. The default is wrong for any specific corpus.
- **You pay embedding cost at ingest with no guaranteed gain.** Every sentence gets encoded, and the boundaries it picks are not provably better than the cheap separators recursive splitting already uses.
- **Independent evaluation pushes back hard.** Chroma's 2024 chunking benchmark (linked at the bottom) ran the threshold-based semantic chunker against a recursive baseline and the recursive baseline was competitive or ahead on the metrics they reported. **The marketing copy is stronger than the evidence.** Their own clustering variant - covered next - did beat the baseline; pure threshold-walking did not.

Pros (theoretical).

- Variable-sized chunks aligned to topic transitions.
- No reliance on author markup.

Cons (in practice).

- Loses to a far cheaper recursive baseline in independent evaluation.
- Adds embedding compute at ingest with no consistent gain.
- Brittle to threshold choice and embedder choice.

When to use. Honestly: only when you have measured a win on your specific corpus against a recursive baseline. The intuition is reasonable; the empirics are not on its side by default.

When NOT to use. As a default. The cost is real and the win is not consistent. If a vendor pitches it without showing you their eval, ask for the eval.

Strategy 7: Cluster Semantic Chunking

Workflow. Variant of semantic chunking that frames the problem as clustering instead of threshold-walking. Embed sentences, cluster them globally to maximise within-cluster similarity, treat the clusters as chunks. Proposed by Chroma in their 2024 chunking evaluation as their own novel method.

Why this beats the threshold-walking approach. Local pairwise similarity is fragile - one bad neighbour pair anywhere and the boundary is wrong. Clustering looks at the global structure of the document and is more robust to the noisy similarity signal that breaks the threshold approach.

Example. A research paper with related-work, methods, results, and discussion sections. Pairwise similarity may flicker as terminology recurs across sections. Clustering looks at the document holistically and groups sentences by their dominant theme.

Pros.

- The semantic variant most worth trying, per Chroma's own evaluation.
- More stable than threshold-walking.
- Still no reliance on author markup.

Cons.

- Still pays the embedding cost at ingest.
- Reference implementation is research code, not yet a stable library API.
- Gains are corpus-dependent. You still have to evaluate.

When to use. If you are committed to a semantic approach and want the variant with the best independent evidence behind it.

When NOT to use. Before you have beaten the recursive baseline on your own eval set. Fix that first. Cluster semantic is a refinement, not a foundation.

Strategy 8: Late Chunking

Workflow. Push the entire document through a long-context embedder first. Then split. Mean-pool the token vectors per chunk to get the chunk embedding. The split happens after the encode, not before.

How it works. In naive chunking, each chunk is embedded in isolation, blind to the rest of the document. With late chunking, every token vector has already attended to the whole document via the transformer's self-attention, so the chunk vectors carry context that the chunks themselves do not contain in their text.

Example. A long Kubernetes doc. A chunk reading "the controller watches for changes and reconciles state" is ambiguous on its own across Deployments, StatefulSets, and custom operators. With late chunking, the token vectors for that sentence already encode "this section is about StatefulSets" because they attended to the surrounding section. **The chunk gets retrieved for the right resource, not the wrong one.**

Pros.

- Resolves cross-chunk references at the embedding level rather than the text level.
- **Gains track document length: the longer the document, the bigger the win.**
- Composes with any chunk-boundary algorithm above. You still pick where to split, you just embed differently.
- The text content of the chunk is unchanged, so storage stays the same.

Cons.

- Requires a long-context embedder. Many embedders still cap at 512 tokens; you need one that can ingest a whole document. Qualifying models include `jina-embeddings-v2` (8K), `nomic-embed-text-v1.5` (8K), OpenAI's `text-embedding-3-large` (8K), and `voyage-3-large` (32K).
- Higher ingest compute - you encode whole documents instead of small chunks.
- On short documents (FAQs, tweets, support emails), the gain shrinks toward zero. The technique cannot help if there is no surrounding context to encode.

When to use. Long continuous prose where chunks reference earlier context: legal contracts ("the Party" defined 30 paragraphs earlier), research papers, manuals, narrative books.

When NOT to use. Short documents. Or when your embedder is 512-token. Or when latency is your binding constraint and you cannot afford the heavier ingest pass.

Strategy 9: Contextual Retrieval

Workflow. Before embedding each chunk, run a cheap LLM pass that prepends a 50 to 100 token "where does this chunk sit in the document" header. Embed the chunk plus the header. Use prompt caching so the full document is cached and only the chunk varies per call - this is what makes the cost survivable.

How it works. The LLM gets the full document and the chunk, and answers a single question: "what context does this chunk need to be retrievable?" The answer is a short situating blurb. You concatenate that blurb to the chunk and embed the whole thing. Same chunk text, very different vector.

Example. A support KB chunk reads "set the timeout to 30 seconds in the advanced settings tab." On its own, useless. After contextualisation: "[Stripe Connect, API v2024-06-20, Webhook configuration] Set the timeout to 30 seconds in the advanced settings tab." Now the embedding encodes product, version, and section. **Same chunk text, very different vector.** The same chunk in a different product's KB gets a different prefix, and the two no longer collide in vector space.

Pros.

- Strong externally validated retrieval gains in the current literature, particularly when combined with hybrid search (BM25 + dense) and a reranker on top. Anthropic reported large failure-rate reductions on their evaluation; results are corpus-dependent so confirm on yours.
- Composable with every other strategy on this list - you can contextualise hierarchical chunks, structure-aware chunks, late-chunked vectors, all of them.
- Reusable across embedders. The enrichment is text, not a vector tied to a specific model.

Cons.

- One LLM call per chunk at ingest. Without prompt caching the bill is brutal; with it, manageable.
- Adds tokens per chunk to your index storage.
- Works best for static or slow-changing corpora. Rapidly changing corpora pay the LLM cost repeatedly.

When to use. Production RAG over heterogeneous corpora where the same phrase recurs with different meanings across documents (multi-product docs, multi-tenant data, multi-version APIs). When retrieval quality is your bottleneck and ingest cost is acceptable.

When NOT to use. Tiny knowledge bases. If your whole corpus fits in the model's context window, just put it all in the prompt with caching - simpler, cheaper, and the LLM does the retrieval implicitly.

How To Pick One

The defensible production stack is rarely a single strategy. It is a layer cake.

1. **Floor:** document structure-aware splitting. Always.
2. **For long hierarchical docs:** add hierarchical retrieval on top.
3. **For long continuous prose:** consider late chunking on top.
4. **For heterogeneous corpora and serious retrieval quality:** add contextual retrieval, hybrid BM25, and a reranker.

Recursive and sentence-based are starting points. Fixed-size is a baseline you compare against. Vanilla semantic chunking is a research curiosity that loses to cheaper alternatives. Cluster semantic is an experiment to run if you have an eval harness. Late chunking and contextual retrieval are the two strategies most likely to move the needle once the floor is in place.

What I Deliberately Left Off: Adaptive Chunking

There is a class of methods that do not pick *a* chunking strategy - they pick *among* strategies per document. The clearest recent example is Ekimetrics' Adaptive Chunking (github.com/ekimetrics/adaptive-chunking, accepted at LREC 2026), which scores candidate chunkings on five intrinsic quality metrics and selects the best one per document.

The intuition is right - no single chunker wins on every document. I am still leaving it off the production-recommendation list, because it has not had a year of independent production usage behind it yet. Revisit this list in a year.

The Only Benchmark That Matters

Every paper, blog, and benchmark you read was measured on someone else's corpus, with someone else's queries. None of them are yours. **Build a golden set of 50 to 200 query-answer pairs from your real usage and measure each strategy against your retriever.** Numbers from a slide deck have a half-life of weeks. Numbers from *your own eval harness* compound.

The chunker is **upstream of everything**. Get it right and the rest of your stack has a chance.

References

- Anthropic: [Introducing Contextual Retrieval](#)
- Jina AI: [Late Chunking in Long-Context Embedding Models](#)
- Late Chunking paper (arXiv): <https://arxiv.org/abs/2409.04701>
- Chroma Research: [Evaluating Chunking Strategies for Retrieval](#)
- Pinecone: [Chunking Strategies for LLM Applications](#)
- LangChain: [Text Splitters API](#)
- LlamaIndex: [Node Parsers / Splitters](#)
- Unstructured.io: [Chunking](#)
- Chonkie: <https://github.com/chonkie-inc/chonkie>
- Greg Kamradt: [5 Levels of Text Splitting](#)

TAGS

CHUNKING Embeddings LLM Llmops RAG

1 Comment



venkatamandavilli

Contributor

3 weeks ago



1 Like

@Mohan-Sharma

This is a very practical take on chunking because it treats it as a production decision, not just a preprocessing trick. I like that the article focuses on what actually survives real usage, since chunking choices can look good in tests but fail once retrieval, attribution, and answer stability matter. It feels especially useful for teams building RAG systems on real enterprise documents, where the structure of the data should drive the strategy.

You must be a registered user to add a comment. If you've already registered, sign in. Otherwise, register and sign in.

[Comment](#)



Top Liked Authors

- | | | |
|---|--|---|
|  |  Swathy_S |  38 |
|  |  Mohan-Sharma |  25 |
|  |  _Dimitri_ |  23 |
|  | venkatamandavilli |  19 |
|  |  yoramfriedman |  18 |
|  | k1000 |  10 |
|  |  DEEPA_DORAIRAJ |  10 |
|  |  Abhishek_Kumar11 |  10 |
|  |  Marla-Ellinger |  7 |
|  | Ahmed_Mosbah |  7 |

[View all](#) >